HeySigApp program specification

October 16, 2026 · Draft for implementation · Android phone and Pixel Watch 3

Purpose and release requirement

Replace the Tasker HeySig3 project with a standalone hearing-aid controller using HeySig's verified direct acoustic commands. Preserve its useful controls, saved settings, noise automation, sleep schedules and safeguards. Remove the everyday dependency on Tasker, AutoInput, AutoVoice, AutoWear and Signia screen automation.

Every user-facing function must work by voice through both the phone and watch. This includes first-time setup, settings changes, confirmations, manual controls, status, saving/restoring, noise calibration and automation, sleep scheduling, diagnostics, cancellation and recovery. A feature is incomplete until its phone and watch voice paths pass acceptance. Screens and touch controls supplement voice. The watch is a first-release requirement.

The phone owns configuration, command execution and acoustic playback. The watch captures commands and presents responses; it does not independently send hearing-aid tones. Users must be told when the phone is disconnected, unavailable, blocked or out of acoustic range of the aids. Voice control cannot grant Android system permissions or bypass secure unlocking: provide spoken directions and resume after the user completes required system actions.

Basis and evidence

The functional baseline is the local `HeySig/HeySig/projects/HeySig3.prj.xml` export and its October 15 user documentation: `HeySig3.md`, `HeySig3-voice-commands.md`, `HeySig3-saved-settings.md`, `HeySig3-noise-settings.md`, `HeySig3-sleep-settings.md`, and `HeySig3-reliability-2026-10-14.md`. The export contains the shared worker, manual controls, voice/watch adapters, noise tasks and sleep/wake profiles.

Repository evidence: Tasker voice, saved settings, noise, sleep, and direct-control code guide. These are baseline observations, not acceptance of the replacement app. Later source notes report an interrupted eight-hour Tasker trial; endurance must be tested afresh.

This specification supersedes the accessibility-first direction in the historical plan. Signia screen readback is not the production state model. Optional external inspection must never silently replace the direct-control path.

Functional parity and changes

HeySig3 capability	HeySigApp requirement
ProgramSwitch / ProgramCurrent	Select configured slots; speak last requested program with its provenance
Volume Set/Up/Down/Max/Mute/Current	Absolute and relative volume; explicit known/unknown state
Balance Set/Up/Down/Soft/Sharp/Current	Sound Sharpness control; accept “balance” as an alias
Tinnitus Set/Up/Down/Max/Mute/Current	Per-program capability checks and tinnitus level
BatteryCheck	Left, right or both audible battery requests; no invented percentage
SaveSettings / RestoreSettings	Durable last-settings record with explicit unknowns and partial-restore results

HeySig3 capability	HeySigApp requirement
VoiceCommand / VoiceRouter / VoiceSettings	Phone and watch voice entry, retry, prompts and results
NoiseSample / Observe / Auto / Stop / Settings	Calibrated phone index, observation and guarded automatic program changes
Override / CancelAutomation / ResumeAutomation	Explicit priority and stop/resume controls
SleepSettings / SleepState / schedules	Voice-configurable programs/times and enabled sleep/wake events
WakeEnable / Disable / Trigger	Optional hands-free “Hey Sig” entry, separately accepted on each device
PhoneAudioCheck / inspection	Spoken readiness, fitting/configuration and bounded diagnostics
AppLaunch / UI calibration / battery page	Replace with HeySig readiness and fitting setup; UI gestures/pages are unnecessary
WatchCommand / WatchNoise	Full watch voice parity; phone microphone is the initial noise source; watch sampling is separate future work

User model and fitting setup

Store one active fitting with a stable ID, revision, qualified address configuration, address-XOR option, validated waveform profile, configured slots, aliases and per-program control capabilities. A capability is supported, unsupported or unknown; unknown tinnitus support requires explicit setup and confirmation before routine use. Never infer tinnitus support solely from a program name.

For this user's initial mapping, offer a reviewable import/template:

Slot	Name
1	Universal
2	Sleep
3	Noisy
4	Outdoor
5	TV
6	Very Noisy

Address 12 with address-XOR enabled worked on this fitting. It is not a universal default. Validate fitting setup with user-directed playback and the user's report of aid response. Never sweep addresses or test maximum settings automatically. The waveform parameters and mappings are documented in the code guide.

Allow voice to list/add/rename/disable configured slots, assign aliases, review addresses and change configuration. Read back a proposed fitting change and obtain explicit confirmation before saving. Enforce slots 1–6, unique names/aliases and valid numeric ranges; reject ambiguous mappings. Renaming a slot must distinguish

a label edit from a change in actual fitting identity. A fitting/slot-identity change invalidates dependent snapshots and disables affected automation until reviewed. A label-only migration preserves slot references and reports the rename.

Import legacy settings through a preview, with source/version validation and explicit voice or touch confirmation. Import program mapping, voice preferences, sleep/noise choices and snapshots only when available and valid. Old gesture coordinates, plugin state and Tasker microphone thresholds are not usable native-app configuration. Preserve the old installation/export for rollback while disabling competing controllers during testing.

State and result contract

Acoustic commands have no aid-state query or receipt acknowledgment. Neither HeySig nor Signia can query actual aid state in this setup. Signia's displayed values can be stale after HeySig sends commands.

Maintain separately:

- Desired request and in-flight operation, with command ID and origin.
- Last fully transmitted request, timestamp, fitting revision and program context.
- Optional user-confirmed response, identified explicitly as user confirmation.
- Unknown or uncertain state after interruption, external edits, aid reset/charging or a declared mismatch.
- Saved restoration record, independent of the current command result.

“Current program” must answer, for example, “Last sent Sleep, program two. Aid state is not verified.” A short “Last sent Sleep” is acceptable after the setup explanation. Unknown state must be spoken as unknown. Never answer “verified,” “connected,” or “battery 80 percent” based solely on playback completion.

Program levels belong to a program context. Selecting a different program must not carry its predecessor's levels into the new program. Retained values for a program remain historical requests, not current readings. Users can say “I changed it in Signia” or “mark settings unknown” to invalidate remembered state. Selecting a program must not automatically apply saved levels unless the user explicitly requests restoration.

Return typed outcomes: transmitted, already requested, busy, blocked, cancelled, failed, partial or unknown. Include target, origin, timestamps, stage and recovery instruction. A storage failure is separate from a successful transmission. No automatic replay after crashes or uncertain completion.

Manual controls

All controls are available on screen and by phone/watch voice:

Control	Range or action	Example phrases
Program	Configured slots 1–6 or unique names	“program two,” “set program to Sleep,” “current program,” “list programs”
Volume	0–15	“volume ten,” “mute,” “maximum volume,” “current volume”
Sound Sharpness	−8 to +7	“balance minus three,” “sharpness zero,” “balance soft,” “balance sharp”
Tinnitus	0–15, supported program	“tin four,” “tinnitus mute,” “maximum tinnitus,” “current tin”

Control	Range or action	Example phrases
Battery	Left, right, both	“check left battery,” “check right battery,” “battery check”
Relative levels	One step from a known program-context request	“volume up,” “balance down,” “tinnitus up”
Save / restore	Last-settings record	“save settings,” “restore settings,” “what is saved?”
Readiness	No aid playback	“check phone audio,” “why is control blocked?”

Preserve HeySig3's absolute phrases, English numbers, minus/negative, set/change/to variants, tin/tinnitus, mode/program, optional sig/sig3 prefix, “program too,” and one trailing sentence punctuation mark. Preserve configured conversation/normal/noisy/very-noisy aliases without hardcoding their destination slots. Match longer names such as Very Noisy before Noisy. Reject unclear or out-of-range commands rather than guessing or clamping.

Relative voice commands extend Tasker voice behavior, whose router did not support them. They require a valid remembered baseline for the relevant program; otherwise ask for an absolute target. Reject movement beyond endpoints. Report that the baseline is remembered.

A repeated request offers “already last requested” without claiming an aid-state match. Provide “send again” to intentionally retransmit it. Battery checks are repeatable actions, not state-setting no-ops. When program context is unknown, require program selection before a level command. Report unsupported tinnitus without transmission.

For both-aid battery checks, run left then right as separate stages with a validated separation. State which requests completed if stopped or failed. Ask the user to listen for the aid indications; optional feedback records reception. Do not wait for an electronic battery reply.

Voice interaction on phone and watch

Use one typed command model and parser for both devices. Recognition and reply adapters may differ; supported functionality must not. Provide a push-to-talk phone microphone button and a watch tile/complication or equivalent accessible launcher. Confirm the launch path works independently of the phone app being visible.

Phone and watch must support multi-turn workflows: “configure noise,” “set sleep time to ten PM,” “rename program two,” “turn spoken results off,” and “restore settings” must be completable with spoken answers and confirmations. Read available options, allow “repeat,” “help,” “back” and “cancel,” and summarize before committing configuration. Ask AM/PM when ambiguous. Expire abandoned dialogs without saving. Do not hold the command executor while waiting for conversation input.

Cancel, empty recognition and recognition errors send no aid command. Unsupported speech offers a fresh retry or cancellation. Never replay the previous phrase on retry. Use the legacy maximum of 20 explicitly chosen attempts and a 60-second unanswered clarification timeout as initial session bounds. A completed control request ends that voice command; settings wizards continue only at explicit next steps.

Keep Voice prompts and Spoken results independent, durable and voice-editable. Both can be off; visible results remain available, and an explicit “read status” request still works. Provide “Sig here” when prompts are on. Replies default to the initiating device: watch commands receive watch feedback, phone commands receive phone feedback. Offer a voice-editable reply-device setting and report unavailable output visibly/haptically. This extends the old watch adapter, which replied through phone TTS.

Coordinate speech capture, TTS, noise sampling and command tones. Suspend sampling/listening during command playback and replies, release microphone ownership reliably, and resume only the previously enabled mode. TTS completion or a fixed delay is not proof of aid receipt. No feedback loop may interpret the app's own tones or speech as environmental noise or a new voice command.

Watch requests carry unique IDs; reconnecting or retrying delivery must not duplicate playback. The phone returns accepted/running/final status, and the watch distinguishes delivery failure from command failure and unknown completion. Stale disconnected requests expire instead of executing later. Phone and watch simultaneous requests share the same executor. Spoken Stop must remain accessible while a long workflow runs.

Optional “Hey Sig” hands-free listening is explicitly enabled/disabled and reports running state. Push-to-talk remains available. Background/locked operation, false triggers, microphone sharing and battery cost require separate phone and watch acceptance. Preserve “Hey Cig” as an optional recognition alias; it does not establish a native wake detector. Do not promise hands-free operation until the chosen implementation passes these gates.

Save and restore

Keep one last-settings record initially, rather than a named-preset library. Store schema version, fitting revision, program slot, known values, capability state, provenance and timestamps. Represent unknown values explicitly; unsupported tinnitus is distinct from unknown tinnitus.

Successful manual setting changes update only the transmitted field and valid same-program information. A first program change cannot read a complete baseline acoustically: record unknown fields and offer voice entry or explicit application of desired values. Queries, failures, repeated-request no-ops and restoration do not overwrite the restore point. Automation does not overwrite the manual restore point; this is an intentional protection beyond the old worker's general save behavior.

“Save settings” saves/reviews HeySig's known requested values. It cannot refresh values from the aids or silently import Signia's displayed state. Offer a voice workflow to enter external changes, labelling them user supplied. State when a record is incomplete.

Restore validates fitting/mapping and previews incomplete fields before transmission. Select the saved program, then send known volume, Sound Sharpness and supported tinnitus values in a serialized sequence. Skip unknown/unsupported values with an explicit summary. Stop on error/cancellation, identify completed and remaining stages, retain the original record and mark uncertain state. Restoration never happens automatically after charging or app startup.

Persist configuration and snapshots atomically; preserve the previous valid record if writing fails. Provide voice-accessible export/import initiation and recovery instructions. Android's destination picker or permission screens may require system interaction.

Noise-driven program selection

Provide Off, Observe and Automatic modes, voice-accessible settings/status, single-sample measurement and Stop. Default off. Initial microphone source is the phone; always identify it. Watch voice controls phone sampling. Watch environmental measurement is outside first-release parity because Tasker had no accepted watch sampler.

Use a documented uncalibrated noise index, never label it sound-pressure dB. Offer a voice-led quiet/normal/loud measurement workflow, showing/speaking sample quality and scale direction. The old Pixel/AutoVoice values −54.5 and −43 are historical measurements, not native-app defaults.

Configure quiet/noisy/very-noisy program roles, low/high boundaries, optional middle band, hysteresis margin, consecutive confirmations (1–20; initial 3), cooldown (initial 60 seconds) and sampling cadence (initial five-second capture followed by ten-second wait). Validate low < high and twice the margin < high − low. Require measured scale-direction confirmation before Automatic. Saving choices stops automation and clears candidates; explicitly start Observe or Automatic afterward.

Define increasing index as louder after any explicit sign conversion. At low boundary L, high boundary H and margin M, enter a louder adjacent band only above boundary + M and a quieter band only below boundary − M; retain the current classified band inside each hysteresis interval. An initial sample classifies by L/H. A jump across both thresholds selects the qualifying outer band. With no middle role, middle-band samples clear the pending target. Count only consecutive fresh qualifying samples for the same target. No switching occurs until the configured count is reached and cooldown permits it.

Count no skipped, stale, interrupted or self-audio samples. Reset candidates after capture failure, a sampling gap beyond the configured freshness limit, manual intervention, cancellation, resume or a blocked/failed transmission. Require fresh confirmation after cooldown and after any gap. Start cooldown from completed program transmission, explicitly unacknowledged by the aids; a repeated already-requested program does not extend it. Use monotonic time for cooldown and sample age.

Observe reports proposed roles/programs but sends nothing. Automatic uses the shared executor and audio guards. Manual program selection enables override; sleep-active and override block automatic noise changes. Show sampling state, source, last sample age, candidate count, last request and blocking reason. Permit one sampling loop only. Stop cancels pending work and prevents any later switch; any capture cleanup remains bounded and visible.

Sleep and wake automation

Voice and touch can choose configured sleep/wake programs and local clock times. Offer 22:00/07:00 as editable suggestions only; require both program selections and unequal times. Saving leaves scheduling off and clears sleep-active suppression. “Enable sleep schedule” arms future events without immediately changing program. “Disable sleep schedule” disarms and clears suppression.

“Sleep” and “wake/awake” submit explicit sleep states only when enabled/configured. “Program Sleep” selects the named slot independently. A sleep selection sets sleep-active only after completed transmission; a wake selection clears it only after completed transmission. State remains an app decision, not sensed sleep or verified aid state. No assumption that the Sleep program mutes microphones or puts aids into standby.

Schedules follow the phone's local timezone. Recompute future events after clock/timezone changes, prevent duplicate execution during repeated clock times, and report skipped or delayed events. Do not replay missed events after restart, permission loss or guard failure. On restart retain choices but leave active automation off pending explicit resume; explain this through phone/watch voice. Any future automatic-rearm mode requires separate acceptance.

Priority, cancellation and audio safeguards

One executor owns command playback, with bounded timeouts and no hidden queue. Reject a new conflicting request as busy. Stop/cancel is accepted during work and interrupts remaining playback/stages; already emitted tones cannot be undone. Do not mark a partially emitted request complete. Never automatically retry an acoustic command.

Priority is: Stop/cancel; manual control; sleep state; noise automation. Manual work does not interrupt an already emitting command unless the user requests Stop; it suppresses pending automatic work. Manual program selection enables override. Override blocks noise and sleep changes. “Override off” releases the block without enabling automation. “Cancel automation” stops modes and clears pending decisions; “resume automation” reviews and explicitly resumes selected valid modes. Sleep-active suppresses noise.

Preserve the current prototype's validated foreground/unlocked and built-in-speaker requirements initially. Preflight checks configured fitting, unlocked phone, DND off, normal audio mode, required ringer state, media at least the configured minimum (initial 50%, rounded up), and no incompatible external output. Do not change phone volume, DND or routing silently. Distinguish aid volume from phone media volume. Watch connectivity/audio routes must be tested with these guards; if watch use exposes an incompatible output route, explain the block rather than disabling the guard.

Signia need not be open. Locked/background acoustic playback is a separate feasibility gate for useful unattended automation and watch operation. Before that gate passes, speak unlock/open-HeySig instructions and mark blocked events as missed. If background playback is accepted later, document the revised conditions and test phone/watch/schedule/noise paths under them. Do not claim unattended program switching while these conditions are unmet.

Screens, persistence and architecture

Provide Home (program and levels, battery, microphone, Stop), Voice, Noise, Sleep, Saved settings, Fitting setup and Diagnostics. All functions exposed here have equivalent phone/watch voice intents and guided spoken workflows. Show provenance and errors in plain language; keep waveform engineering details in fitting diagnostics.

The shared Kotlin core owns commands, parsing, fitting models, state provenance, snapshot validation, noise decisions, priority and scheduling rules. Android owns UI, audio, microphone, speech, persistence, permissions and lifecycle. A Wear OS companion owns voice capture, replies and reliable request delivery to the phone. Keep the validated renderer behind a transport interface; migrate research code with frozen waveform regression checks. Use a normal reproducible Android/Wear build before release.

Version storage schemas. Persist fitting/aliases, voice settings, noise calibration/settings, schedule settings and restore records. Active workers/listeners are runtime state; a process restart must not falsely report them running. Store bounded numeric diagnostics by default, without raw microphone recordings. Include command stages, sampler gaps and interruption reasons; allow voice to explain/export diagnostics and delete them. No speech or aid-control cloud service is required by the spec; disclose any network-dependent recognition implementation during setup.

Implementation and acceptance gates

1. Establish shared models, durable fitting setup and serialized direct transport. Preserve packet and PCM regressions. Build manual controls plus phone/watch voice together.
2. Complete all setup, settings and recovery voice workflows on both devices. Verify independent watch launch, local replies, disconnect handling and duplicate suppression.
3. Add durable save/restore and provenance-aware queries. Verify partial failures, program-context changes, outside edits and restart persistence.
4. Calibrate the native phone sampler; add Observe before Automatic. Validate hysteresis, gaps, cooldown, override and sleep interactions.

5. Add sleep scheduling and lifecycle recovery. Establish the locked/background playback gate before claiming unattended automation.
6. Accept optional hands-free wake independently, then run endurance and release checks. Keep the existing Tasker export available until parity is accepted.

For **every functional row and settings workflow**, maintain phone-touch, phone-voice and watch-voice acceptance cases. Test recognized intent and actual end-to-end capture/delivery/result separately. A parser unit test or watch delivery acknowledgment alone does not pass voice acceptance. Include ambiguous speech, cancellation, timeout, busy state and blocked-audio recovery.

Required physical tests include configured program beep feedback (Sleep two, TV five on this fitting), both battery indications, representative volume/Sound Sharpness/tinnitus changes and restoration. Preserve tested ranges and unsupported-program cases; do not infer all model/firmware compatibility. Record playback completion separately from user response.

Required logic/lifecycle tests include unknown baselines, repeated request/resend, snapshot write failure, interrupted restore, invalid mappings, duplicate watch messages, loss of phone connection, microphone contention, self-audio exclusion, noise boundary oscillation, large sample gaps, sleep/override priority, changed timezones, restart, permission revocation and no-ADB operation.

Run eight-hour observation and automatic-mode trials with explicit stop/restart detection, sample gaps, bounded memory and battery measurements; compare an equivalent idle baseline before attributing battery use to sampling. Test wake listening separately and then in supported combinations. A crashed or stale run is interrupted, not completed.

Release requires no Tasker/plugins for accepted functions, successful fresh install/upgrade/configuration recovery, both voice paths complete, clear known limitations, and evidence for each advertised operating condition. iPhone and direct watch-to-aid playback remain separate future projects.